

Assignment 3: End-to-End ML Versioning

Part E: Simulated Collaboration & Conflict Resolution

Student: Munam **Repository:** github.com/Munam32/MLOps **Branches:** main, teammate-sim

The scenario

- Setup** dev (pipeline v1 and v2) was first merged into main. Then two "contributors" changed the same normalization lines in preprocess.py in different ways, and each regenerated the data.
- Pointer file** In this project data/processed is a pipeline output, so its hash lives in dvc.lock rather than a separate .dvc file. dvc.lock therefore plays the role of the pointer file, and it is where the data conflict appears.

E1 Teammate's change: scale to [-1, 1]

Goal: on a branch from main, change normalization, regenerate the data, and commit.

On teammate-sim, pixels are divided by 127.5 and shifted down by 1, so they range from -1 to 1 instead of 0 to 1. dvc repro regenerated data/processed (new hash bc407e...) and retrained the model.

teammate-sim: the change

```
$ git diff src/preprocess.py
diff --git a/src/preprocess.py b/src/preprocess.py
index 0631167..f5f735f 100644
--- a/src/preprocess.py
+++ b/src/preprocess.py
@@ -20,12 +20,13 @@ def main():
     train_raw = np.load(RAW_DIR / "train.npz")
     test_raw = np.load(RAW_DIR / "test.npz")

-    train_images = train_raw["images"].astype("float32") / 255.0
+    # Scale pixels from [0, 255] to [-1, 1] so inputs are centred on zero
+    train_images = train_raw["images"].astype("float32") / 127.5 - 1.0
     train_labels = train_raw["labels"]
-    test_images = test_raw["images"].astype("float32") / 255.0
+    test_images = test_raw["images"].astype("float32") / 127.5 - 1.0
     test_labels = test_raw["labels"]

-    assert 0.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [0, 1]"
+    assert -1.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [-1, 1]"

     train_images, val_images, train_labels, val_labels = train_test_split(
         train_images,
```

Regenerate and commit

```
$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to ~/Desktop/MLOps/data/processed
Updating lock file 'dvc.lock'
Running stage 'train':
> python src/train.py
Saved trained model and history to ~/Desktop/MLOps/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3669, test accuracy: 0.8832
Saved metrics.json to ~/Desktop/MLOps and confusion_matrix.png to ~/Desktop/MLOps/reports
Updating lock file 'dvc.lock'

$ grep -A3 'path: data/processed' dvc.lock | head -4
- path: data/processed
  hash: md5
  md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir
  size: 219591530

$ git commit -q -m 'feat: scale pixels to [-1, 1] in preprocess' -m 'Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>' && git log --oneline -1
075fb5f feat: scale pixels to [-1, 1] in preprocess
```

E2 Main's change: z-score standardization

Goal: on `main`, change the same lines differently.

After switching back, `dvc checkout` restored `main`'s own data. Then pixels were standardized with the training set's mean and standard deviation. `dvc repro` produced a different hash (`cf42c4...`).

```
main: a different change to the same lines

$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 25 commits.
  (use "git push" to publish your local commits)

$ dvc checkout
M      data/processed/
M      models/

$ dvc status
Data and pipelines are up to date.

$ git diff src/preprocess.py
diff --git a/src/preprocess.py b/src/preprocess.py
index 0631167..45feaad 100644
--- a/src/preprocess.py
+++ b/src/preprocess.py
@@ -20,12 +20,15 @@ def main():
     train_raw = np.load(RAW_DIR / "train.npz")
     test_raw = np.load(RAW_DIR / "test.npz")

-    train_images = train_raw["images"].astype("float32") / 255.0
+    # Standardize with the training set's mean and std (z-score)
+    train_images = train_raw["images"].astype("float32")
+    mean, std = train_images.mean(), train_images.std()
+    train_images = (train_images - mean) / std
     train_labels = train_raw["labels"]
-    test_images = test_raw["images"].astype("float32") / 255.0
+    test_images = (test_raw["images"].astype("float32") - mean) / std
     test_labels = test_raw["labels"]

-    assert 0.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [0, 1]"
+    assert abs(train_images.mean()) < 1e-3, "training pixels not centred on 0"

     train_images, val_images, train_labels, val_labels = train_test_split(
         train_images,
```

Regenerate and commit

```
$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to ~/Desktop/ML0ps/data/processed
Updating lock file 'dvc.lock'
Running stage 'train':
> python src/train.py
Saved trained model and history to ~/Desktop/ML0ps/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3922, test accuracy: 0.8807
Saved metrics.json to ~/Desktop/ML0ps and confusion_matrix.png to ~/Desktop/ML0ps/reports
Updating lock file 'dvc.lock'

$ grep -A3 'path: data/processed' dvc.lock | head -4
- path: data/processed
  hash: md5
  md5: cf42c4dbfcff8dd7a675d8a0b389870d.dir
  size: 219591530

$ git commit -q -m 'feat: standardize pixels with z-score in preprocess' -m 'Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>' && git log --oneline -1
43e7a8d feat: standardize pixels with z-score in preprocess
```

E3 Merge: code conflict and data conflict

Goal: merge `teammate-sim` into `main` and see both kinds of conflict.

```
git merge teammate-sim
```

```
$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging metrics.json
CONFLICT (content): Merge conflict in metrics.json
warning: Cannot merge binary files: reports/confusion_matrix.png (HEAD vs. teammate-sim)
Auto-merging reports/confusion_matrix.png
CONFLICT (content): Merge conflict in reports/confusion_matrix.png
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
$ git status --short
UU dvc.lock
UU metrics.json
UU reports/confusion_matrix.png
UU src/preprocess.py
```

Code conflict: src/preprocess.py

```
$ sed -n '20,40p' src/preprocess.py
train_raw = np.load(RAW_DIR / "train.npz")
test_raw = np.load(RAW_DIR / "test.npz")

<<<<<<< HEAD
# Standardize with the training set's mean and std
(z-score)
train_images = train_raw["images"].astype("float32")
mean, std = train_images.mean(), train_images.std()
train_images = (train_images - mean) / std
train_labels = train_raw["labels"]
test_images = (test_raw["images"].astype("float32")
- mean) / std
test_labels = test_raw["labels"]

assert abs(train_images.mean()) < 1e-3, "training pi
xels not centred on 0"
=====
# Scale pixels from [0, 255] to [-1, 1] so inputs ar
e centred on zero
train_images = train_raw["images"].astype("float32")
/ 127.5 - 1.0
train_labels = train_raw["labels"]
test_images = test_raw["images"].astype("float32") /
127.5 - 1.0
test_labels = test_raw["labels"]

assert -1.0 <= train_images.min() and train_images.m
ax() <= 1.0, "pixels not in [-1, 1]"
>>>>>>> teammate-sim
```

Data conflict: dvc.lock

```
$ grep -n -B2 -A12 '<<<<<<<' dvc.lock | head -60
24- - path: src/preprocess.py
25- hash: md5
26:<<<<<<< HEAD
27- md5: 2fe971c6999537f883ba67ce0644c5c7
28- size: 1746
29-=====
30- md5: 61027e084af8221bfc215d0f76c189c8
31- size: 1679
32:>>>>>>> teammate-sim
33- params:
34- params.yaml:
35- preprocess.seed: 42
36- preprocess.val_size: 0.1
37- outs:
38- - path: data/processed
39- hash: md5
40:<<<<<<< HEAD
41- md5: cf42c4dbfcff8dd7a675d8a0b389870d.dir
42-=====
43- md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir
44:>>>>>>> teammate-sim
45- size: 219591530
46- nfiles: 3
47- train:
48- cmd: python src/train.py
49- deps:
50- - path: data/processed
51- hash: md5
52:<<<<<<< HEAD
53- md5: cf42c4dbfcff8dd7a675d8a0b389870d.dir
54-=====
55- md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir
56:>>>>>>> teammate-sim
```

Code conflict (left): both versions of the normalization block, between <<<<<<< HEAD (main) and >>>>>>> teammate-sim. **Data conflict (right):** dvc.lock records two different hashes for the same path, data/processed. The models hash conflicts too, since each model was trained on different data. metrics.json and the plot conflict as a knock-on effect.

Knock-on conflict: metrics.json

```
$ cat metrics.json
{
<<<<<<< HEAD
"test_loss": 0.3922438621520996,
"test_accuracy": 0.8806999921798706
=====
"test_loss": 0.36694392561912537,
"test_accuracy": 0.8831999897956848
>>>>>>> teammate-sim
}
```

E4 Resolve both conflicts

Goal: reconcile the code, pick the authoritative data, and sync the working directory.

BRANCH	NORMALIZATION	DATA/PROCESSED HASH	TEST ACCURACY	TEST LOSS
main	z-score	cf42c4...	88.07%	0.3922
teammate-sim	[-1, 1]	bc407e...	88.32%	0.3669

Code. Rather than discard one approach, I kept both: `preprocess.py` now reads a new `preprocess.normalization` setting from `params.yaml` (`minus_one_one` or `zscore`), and `dvc.yaml` lists it as a param so changing it re-runs the pipeline.

Resolved code: both methods, chosen by `params.yaml`

```
$ sed -n '23,40p' src/preprocess.py
train_images = train_raw["images"].astype("float32")
train_labels = train_raw["labels"]
test_images = test_raw["images"].astype("float32")
test_labels = test_raw["labels"]

if params["normalization"] == "minus_one_one":
    # Scale pixels from [0, 255] to [-1, 1] so inputs are centred on zero
    train_images = train_images / 127.5 - 1.0
    test_images = test_images / 127.5 - 1.0
    assert -1.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [-1, 1]"
elif params["normalization"] == "zscore":
    # Standardize with the training set's mean and std
    mean, std = train_images.mean(), train_images.std()
    train_images = (train_images - mean) / std
    test_images = (test_images - mean) / std
    assert abs(train_images.mean()) < 1e-3, "training pixels not centred on 0"
else:
    raise ValueError(f"unknown normalization: {params['normalization']}")

$ head -4 params.yaml
preprocess:
  val_size: 0.1
  seed: 42
  normalization: minus_one_one # or zscore
```

Data. The teammate version scored better on both metrics, so it is authoritative and `minus_one_one` is the default. I took that side's `dvc.lock`, metrics and plot with `git checkout --theirs`, then ran `dvc checkout` so the files on disk match its hash.

Resolved data: teammate-sim version is authoritative

```
$ git checkout --theirs dvc.lock metrics.json reports/confusion_matrix.png
Updated 3 paths from the index

$ grep -A2 'path: data/processed' dvc.lock | head -3
- path: data/processed
  hash: md5
  md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir

$ dvc checkout
M    data/processed/
M    models/

$ git status --short
M dvc.lock
M dvc.yaml
M metrics.json
M params.yaml
M reports/confusion_matrix.png
M src/preprocess.py

$ dvc status
preprocess:
  changed deps:
    modified:      src/preprocess.py
    params.yaml:
      new:          preprocess.normalization
```

`dvc status` still flags `preprocess`, because the merged code differs from both originals. Re-running it produced **exactly the same hash** as the teammate version (`bc407e...`), so DVC skipped `train` and `evaluate`. This confirms the reconciled code reproduces the chosen data.

Re-run the changed stage

```

$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to ~/Desktop/ML0ps/data/processed
Updating lock file 'dvc.lock'
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping

$ grep -A2 'path: data/processed' dvc.lock
- path: data/processed
  hash: md5
  md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir

```

E5 Clean state, merge commit, reproducible

Goal: confirm a clean state, finish the merge, and prove `dvc repro` works from it.

Finish the merge and prove reproducibility

```

$ dvc status
Data and pipelines are up to date.

$ git add dvc.lock

$ git commit -q -m 'Merge branch teammate-sim: reconcile normalization' -m 'Keeps both methods behind preprocess.normalization. minus_one_one is the default: 88.32% test accuracy vs 88.07% for zscore.' -m 'Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>' && git log --oneline -1
b0450f8 Merge branch teammate-sim: reconcile normalization

$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Data and pipelines are up to date.

$ dvc status
Data and pipelines are up to date.

$ git status --short

$ git log --oneline --graph --decorate -6
* b0450f8 (HEAD -> main) Merge branch teammate-sim: reconcile normalization
| \
| * 075fb5f (teammate-sim) feat: scale pixels to [-1, 1] in preprocess
* | 43e7a8d feat: standardize pixels with z-score in preprocess
| /
* ca989a1 (dev) Merge branch dev: pipeline v1 and v2
| \
| * 5301455 (origin/dev) docs: add Part D report
| * c63beae (tag: v2) exp: increase dense_units from 128 to 256

```

Push data (Google Drive) and code (GitHub)

```

$ dvc push --all-commits
WARNING: failed to collect '0f2e2296f47902a5f1e560248713db1baa865313', skipping
WARNING: failed to collect 'da2542a5cd4b851d196a47e78c152f31b9e42c1a', skipping
... 19 more of these, for commits made before DVC was set up
Collecting
Pushing
14 files pushed

$ git push origin main dev teammate-sim
To github.com:Munam32/ML0ps.git
  5301455..ca989a1 dev -> dev
  bdd6d6f..b0450f8 main -> main
  * [new branch] teammate-sim -> teammate-sim

```

`--all-commits` pushes the data for every version in the history, including `main`'s z-score version from E2, which is no longer at the tip of any branch. The warnings are commits from before `dvc init`, which have no DVC data to collect.

In short: Git resolves code conflicts line by line, but it can't merge data. It can only show two hashes. The data conflict is resolved by choosing a version (here, by comparing metrics), restoring it with `dvc checkout`, and letting `dvc repro` confirm the pipeline agrees. After the merge, `dvc repro` skips all four stages: the merged state is fully reproducible.